

Internet & Web Fundamentals — Comprehensive Active Recall Paper

Kaise use karein: Pehle poora "Concept Notes" section parh lein. Uske baad "Active Recall Question Bank" section ko akele attempt karein (answers na dekhein). Jab sab try ho jaye, "Answer Key" se check karein. Jo bhi galat ho, wapas concept notes wale section pe jayein.

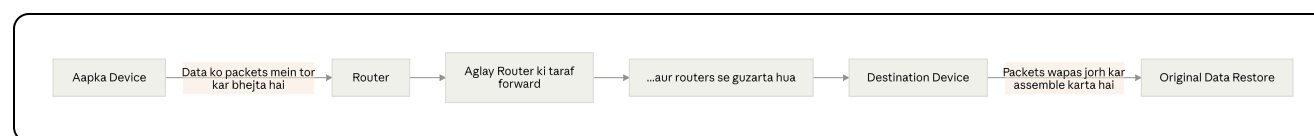
PART 1 — CONCEPT NOTES

1. Internet Kya Hai — Bunyadi Tasawwur (Fundamentals)

Network ek group hai computers/devices ka jo aapas mein connected hon. Jab bohat saare alag-alag networks aapas mein connect ho jate hain, tou wo bun jata hai **Internet**. Isi liye kaha jata hai: *"Internet is a network of networks."*

Internet 1960s ke late mein US Department of Defense ne banaya tha — maqsad tha ek **decentralized** communication network banana jo nuclear attack jaisi situation mein bhi survive kar sake.

Internet kaam kaise karta hai (High Level Flow)



- Jab aap data bhejte hain, wo chote **packets** (jaisay puzzle pieces) mein tor diya jata hai.
- Ye packets physical infrastructure (fiber optic cables, modems) se guzar kar routers ke through destination tak pohanchte hain.

- Destination pe pohanch kar sab packets wapas assemble ho kar original data ban jate hain.

Core Protocols

Protocol	Kaam
IP (Internet Protocol)	Har device ko unique IP address deta hai aur packets ko sahi destination tak route karta hai
TCP (Transmission Control Protocol)	Data ko packets mein todta hai, reliable aur correct order mein delivery ensure karta hai
UDP	TCP jaisa hi lekin reliability guarantee nahi karta — faster hota hai
DNS	Domain names ko IP addresses mein translate karta hai
HTTP/HTTPS	Client aur server ke darmiyan data transfer karta hai
SSL/TLS	Internet communication ko encrypt/secure karta hai

Important distinction: Internet = Hardware (network of machines). World Wide Web (WWW) = Software jo internet ke upar chalta hai, jahan HTTP se web pages access hoti hain.

Bunyadi Terminology

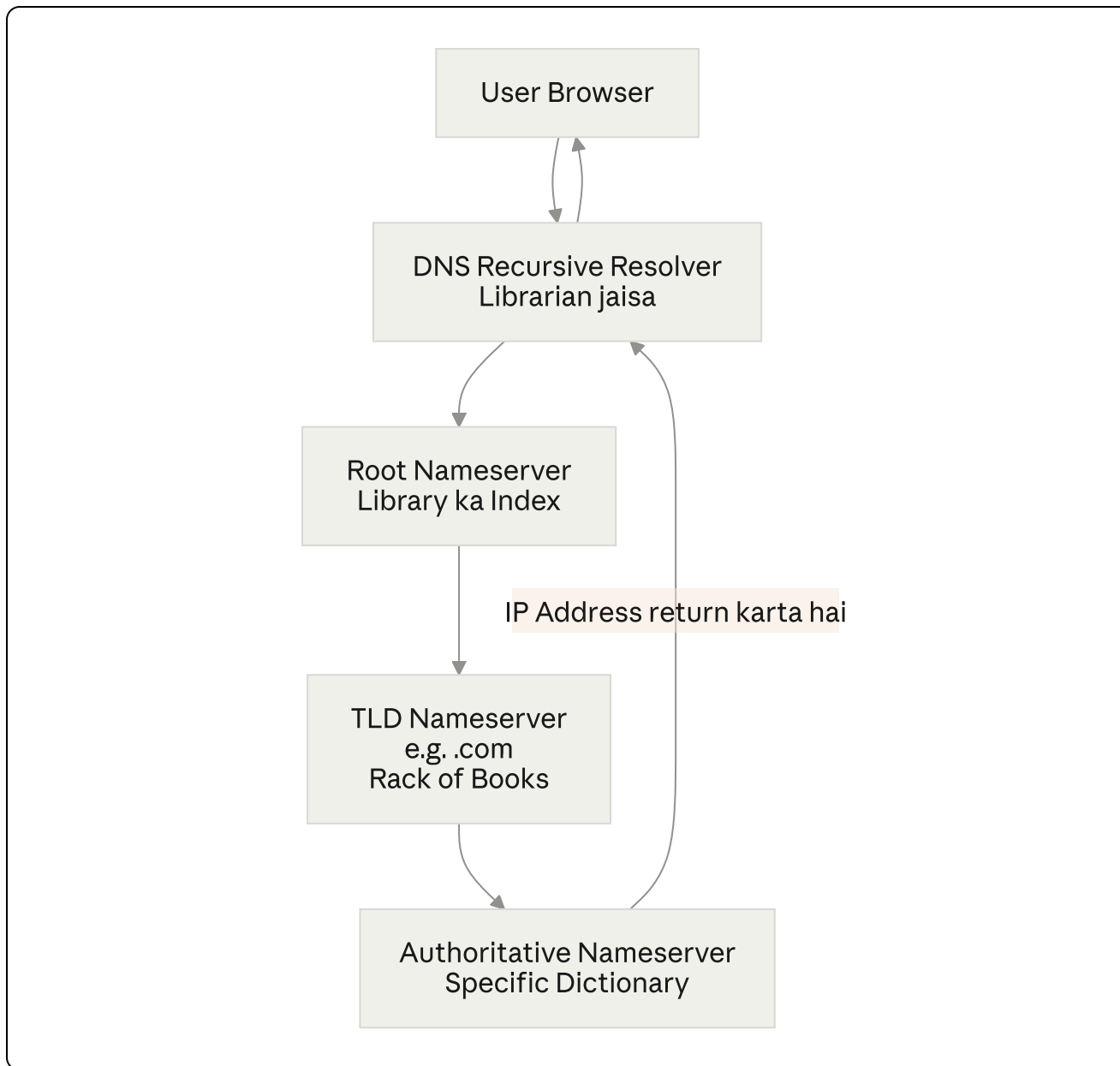
- **Packet:** Data ka chota unit jo transmit hota hai.
- **Router:** Device jo packets ko different networks ke darmiyan direct karta hai.
- **IP Address:** Har device ka unique identifier (e.g. `192.168.1.1`).
- **Domain Name:** Human-readable naam (e.g. `google.com`).
- **DNS:** Domain names ko IP address mein translate karne wala system.

Ports, Sockets, Connections (TCP/IP Applications)

- **Port:** Device pe chal rahe kisi application/service ko identify karta hai (unique number).
 - **Socket:** IP address + Port number ka combination — ek specific communication endpoint.
 - **Connection:** Do sockets ke darmiyan establish hoti hai jab dono devices communicate karna chahte hain. Is process mein parameters negotiate hote hain (max segment size, window size).
 - **Data Transfer:** Connection establish hone ke baad data **segments** mein transfer hota hai, har segment ka sequence number hota hai reliable delivery ke liye.
-

2. DNS (Domain Name System) — Detailed

DNS ko "**phonebook of the internet**" kaha jata hai. Ye human-readable domain names (jaise `nytimes.com`) ko machine-friendly IP addresses mein convert karta hai.



1. **DNS Recursor (Recursive Resolver):** Client se query leta hai aur uski taraf se additional requests karta hai — librarian ki tarah jo book dhoondhta hai.
2. **Root Nameserver:** Resolution ka pehla step — library ka index jaisa, aage kahan jana hai batata hai.

3. **TLD Nameserver:** Domain ka last portion handle karta hai (`.com`, `.net`) — library ka specific rack.
4. **Authoritative Nameserver:** Final stop — actual IP address yahan se milta hai, dictionary jaisa jo exact meaning deta hai.

Recursive Resolver vs Authoritative Server

- **Recursive Resolver:** Query ke **start** pe hota hai — record dhoondhne ke liye multiple requests karta hai, caching bhi karta hai taake baar baar poora lookup na karna pare.
- **Authoritative Server:** Query ke **end** pe hota hai — record ka actual owner/source of truth. Kisi aur server se query nahi karta, khud jawab deta hai.
- Agar query kisi subdomain (jaise `blog.cloudflare.com`) ke liye ho, tou authoritative nameserver ke baad ek aur nameserver add hota hai jo CNAME record store karta hai.

8 Steps of a DNS Lookup (Full Uncached Flow)

1. User browser mein `example.com` type karta hai → query recursive resolver tak jati hai.
2. Resolver **root nameserver** ko query karta hai.
3. Root server TLD server (`.com`) ka address deta hai.
4. Resolver `.com` TLD ko request karta hai.
5. TLD server domain ke nameserver ka IP deta hai.
6. Resolver domain ke authoritative nameserver ko query karta hai.
7. Authoritative nameserver IP address return karta hai.
8. Resolver browser ko IP address de deta hai.

Uske baad: Step 9 — browser us IP pe HTTP request bhejta hai. Step 10 — server webpage return karta hai.

Note: Agar record cached ho (browser cache ya OS cache mein), tou steps skip ho jate hain aur lookup fast ho jata hai.

DNS Caching Layers

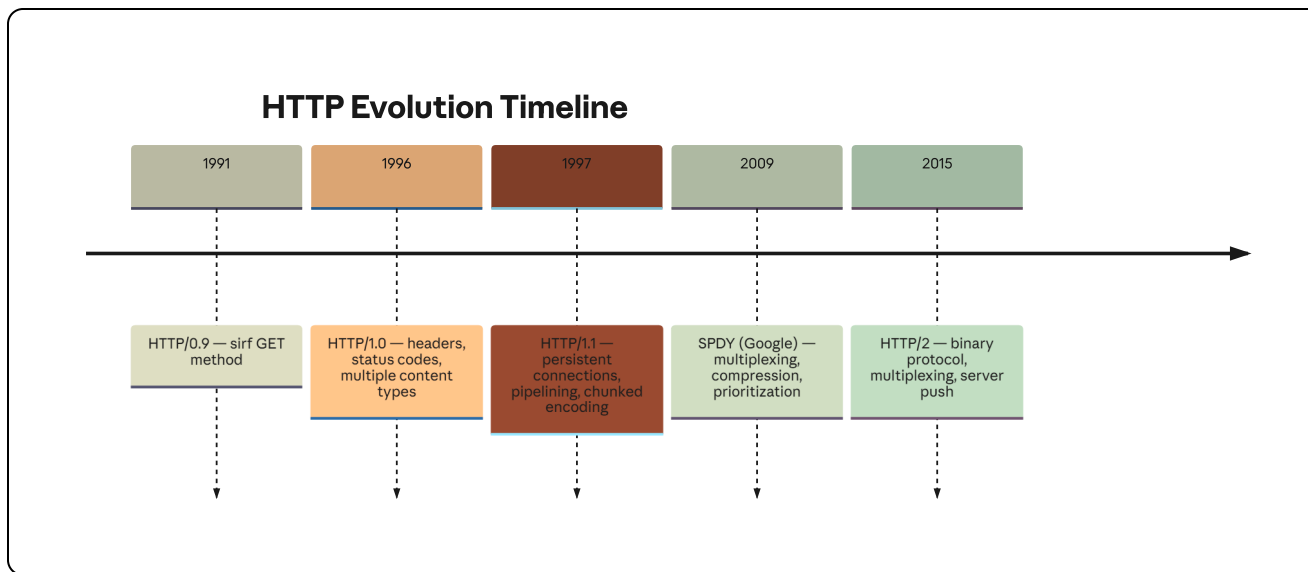
1. **Browser DNS Cache:** Sabse pehle check hota hai (Chrome mein: `chrome://net-internals/#dns`).
2. **OS-level Caching (Stub Resolver):** Agar browser cache mein nahi mila, OS ka "stub resolver" apna cache check karta hai, phir ISP ke recursive resolver ko query bhejta hai.
3. **ISP Recursive Resolver Cache:** Agar A record na ho lekin NS record ho, seedha authoritative nameserver ko query karta hai (root aur TLD skip). Agar NS bhi na ho, TLD ko query karta hai. Agar wo bhi na ho, root se start karta hai.

3 Types of DNS Queries

Query Type	Kya Hota Hai
Recursive Query	Client server se demand karta hai ke ya poora answer do ya error do
Iterative Query	Server best possible answer deta hai, agar match nahi tou referral deta hai agle server ka
Non-Recursive Query	Server ke paas record already available hota hai (authoritative ya cache se)

3. HTTP — Complete Evolution (0.9 se HTTP/2 tak)

HTTP ek TCP/IP-based **application layer** protocol hai jo define karta hai ke client aur server kaise communicate karein. Default TCP port: **80** (HTTP), **443** (HTTPS).



HTTP/0.9 (1991) — "The One Liner"

- Sirf ek method: **GET**
- Koi headers nahi
- Response sirf HTML ho sakta tha
- Request ke baad turant connection close ho jata

HTTP/1.0 (1996)

- Naye methods add hue: **POST, HEAD**
- **Headers** add hue (request aur response dono mein)
- **Status codes** introduce hue
- Multiple content types support hui (images, video, plain text)
- Character sets, caching, authorization, content encoding add hue
- **Major drawback:** Har request ke liye **naya TCP connection** khulta tha — 1 webpage jisme 20 items (images, CSS, JS) hon, uske liye 20 separate connections lagte the → bohat slow (har connection ke liye three-way handshake overhead)

1. **SYN** — Client random number (x) bhejta hai
2. **SYN-ACK** — Server apna random number (y) aur $(x+1)$ bhejta hai
3. **ACK** — Client $(y+1)$ bhej kar confirm karta hai

Kuch HTTP/1.0 implementations ne `Connection: keep-alive` header try kiya (connection open rakhne ke liye) lekin widely supported nahi tha.

HTTP is also **stateless** — server client ke baare mein kuch record nahi rakhta, har request independent honi chahiye (apni info khud carry kare).

HTTP/1.1 (1997)

Bade improvements:

- **Naye methods:** PUT, PATCH, OPTIONS, DELETE
- **Host header mandatory** ho gaya
- **Persistent Connections:** Ab default mein connection close nahi hoti — multiple sequential requests same connection pe ja sakti hain. Close karne ke liye `Connection: close` header bhejna hota hai.
- **Pipelining:** Client multiple requests bhej sakta hai bina response ka wait kiye (same connection pe), server responses **same sequence** mein bhejta hai. Client ko pata chalta hai response kahan khatam hua via `Content-Length` header.
- **Chunked Transfer Encoding:** Jab content-length pehle se pata nahi hota (dynamic content), server data chunks mein bhejta hai, har chunk apna length batata hai, aakhir mein empty chunk (length 0) transmission complete signal karta hai. Header: `Transfer-Encoding: chunked`.
- Digest aur proxy authentication add hui
- Caching, byte ranges, character sets, language negotiation, client cookies, compression, naye status codes

HTTP/1.1 ki limitation: Ek waqt mein sirf **ek outstanding connection** ho sakta tha per TCP connection. Pipelining ne isay fix karne ki koshish ki lekin **Head-of-Line (HOL) Blocking** ka masla raha — agar ek slow/heavy request pipeline mein phans jaye, uske peeche wali sab requests wait karti hain.

Developers ne workarounds nikale: sprite sheets, base64-encoded images in CSS, giant bundled CSS/JS files, domain sharding.

SPDY (2009) — Google ka Experiment

- Core idea: **Latency kam karo, bandwidth badhane se performance utni nahi badhti jitni latency kam karne se badhti hai**
- Features: multiplexing, compression, prioritization, security
- HTTP ko replace nahi karta tha — ek **translation layer** tha application layer pe
- 2015 mein Google ne SPDY ko HTTP mein merge kar diya → **HTTP/2** ban gaya, SPDY deprecated ho gaya

HTTP/2 (2015)

Key differences from HTTP/1.1:

1. **Binary Protocol (not textual)** — Parse karna easier hai lekin human-readable nahi raha.
 - **Frames:** HTTP messages ab ek ya zyada frames se milkar bante hain (HEADERS frame for metadata, DATA frame for payload, RST_STREAM, SETTINGS, PRIORITY etc.)
 - **Streams:** Frames ka collection, har request/response ko unique **Stream ID** milta hai. Client-initiated requests **odd numbers**, server responses **even numbers** use karte hain.
 - **RST_STREAM:** Special frame jo client use karta hai kisi stream ko abort karne ke liye bina poori connection close kiye (HTTP/1.1 mein sirf poori connection close karke hi rok sakte the).

2. **Multiplexing:** Ek hi TCP connection pe sab streams **asynchronously** bhejay jate hain — koi naya connection nahi khulta. Response bhi asynchronous order mein aata hai, stream ID se identify hota hai kaunsa packet kis stream ka hai. Isse **HOL blocking** solve ho jata hai.
3. **Header Compression (HPACK):** Repetitive headers (jaise user-agent, cookies) baar baar poora nahi bheje jate. Literal values **Huffman coding** se encode hoti hain, aur client-server dono ek **headers table** maintain karte hain jisse repetitive headers ko reference kiya ja sakta hai. Naye pseudo-headers add hue: `:method`, `:scheme`, `:host`, `:path`.
4. **Server Push:** Server ko andaza hota hai client kya maangega, tou wo **PUSH_PROMISE** frame bhej kar bina client ke maangay resource pehle hi bhej deta hai — roundtrips kam hoti hain.
5. **Request Prioritization:** Client `HEADERS` frame mein ya alag `PRIORITY` frame se stream ki priority set kar sakta hai. Server us priority ke hisab se resources allocate karta hai.
6. **Security:** Spec mein TLS mandatory nahi hai lekin practically sab vendors sirf TLS ke sath HTTP/2 support karte hain (TLS 1.2+ required agar use ho).

Quick Comparison Table

Feature	HTTP/1.0	HTTP/1.1	HTTP/2
Connections	1 request per connection	Persistent connections	Single multiplexed connection
Format	Text	Text	Binary
Multiple requests	Nahi	Pipelining (HOL blocking issue)	True Multiplexing
Headers	Uncompressed	Uncompressed	HPACK compressed

Feature	HTTP/1.0	HTTP/1.1	HTTP/2
Push	Nahi	Nahi	Server Push

4. Web Hosting

Web hosting = infrastructure jo aapki website ko internet pe available rakhti hai (computer + network infra), aur email jaisi services bhi deti hai.

Hosting Types

- **Shared Hosting** — sabse popular aur affordable, ek server multiple websites host karta hai
- **Reseller Hosting**
- **VPS (Virtual Private Server)**
- **Dedicated Server**

Kaise Kaam Karta Hai

- **Server:** Powerful computer jo **data center** mein rakha hota hai (secure, multi-million pound facility)
- **Data Center provide karta hai:** network + power connectivity, temperature control, backup systems, fire suppression, physical security
- **Server-level redundancy:** critical components (hard drives, power supply units) **dual** hote hain — availability ensure karne ke liye

Jab aap hosting **kharidte** hain, aap actually kharid rahe hote hain:

- **Disk space** (aapki website files, emails)
- **Bandwidth** (monthly traffic allowance — email traffic + website visitors)

Control Panel

- Popular tool: **cPanel** — email setup, domain management, database management jaisi cheezein manage karta hai bina technical complexity ke.
- **Website Builder:** Non-technical users ke liye templates-based tool.
- Advanced users ke liye: CMS (WordPress), e-commerce tools (PrestaShop, Magento)

Security & Migration

- **Security features:** anti-spam software, data backup
 - **Migration:** Ek hosting provider se dusre pe shift hona — files copy hoti hain, domain/nameserver settings mein chotay changes karne padtay hain
-

5. Browser Internals — Browser Andar Kya Karta Hai

Browser ke Main Components

Component	Kaam
User Interface	Address bar, back/forward, bookmarks (jo bhi page window ke ilawa nazar aata hai)
Browser Engine	UI aur Rendering Engine ke darmiyan actions ko marshal karta hai
Rendering Engine	Requested content (HTML/CSS) parse aur display karta hai
Networking	HTTP requests jaisay network calls handle karta hai
UI Backend	Basic widgets draw karta hai (combo boxes, windows)

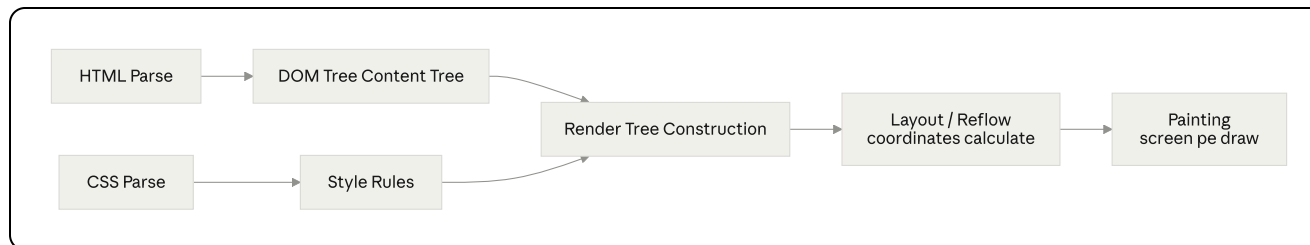
Component	Kaam
JavaScript Interpreter	JS code parse aur execute karta hai
Data Storage	Persistence layer — cookies, localStorage, IndexedDB waghera

Chrome jaisay browsers har tab ke liye rendering engine ka **alag instance** chalatay hain (separate process per tab).

Rendering Engines by Browser

Browser	Engine
Internet Explorer	Trident
Firefox	Gecko
Safari	WebKit
Chrome, Opera (v15+)	Blink (WebKit ka fork)

Main Rendering Flow



1. HTML parse ho kar **DOM nodes** ka tree banta hai ("content tree")
2. External CSS files aur `<style>` tags parse hoti hain

3. DOM + Style info milkar **Render Tree** banata hai (visual rectangles, sahi order mein)
4. **Layout** stage — har node ko exact screen coordinates milte hain
5. **Painting** stage — render tree traverse ho kar UI backend se paint hota hai

Ye process **gradual** hota hai — poora HTML parse hone ka wait nahi karta, jitna content aata hai utna parse+display hota rehta hai.

Terminology difference: Firefox render tree ko "**Frame Tree**" kehta hai (elements = frames), WebKit "**Render Tree**" (elements = Render Objects). WebKit "layout" bolta hai, Gecko "**Reflow**" bolta hai.

Parsing — General Concepts

- **Parsing** = document ko ek structure (tree) mein translate karna jo code use kar sake, jisay **parse tree/syntax tree** kehte hain.
- **Grammar:** Vocabulary + syntax rules jo language define karti hain — agar poori tarah BNF mein express ho sakay, tou usay **context-free grammar** kehte hain.
- **Lexer (Tokenizer):** Input ko **tokens** mein todta hai (vocabulary ke valid pieces)
- **Parser:** Tokens se **parse tree** banata hai grammar rules follow karke

Do types of parsers:

- **Top-down:** High-level structure se start, phir rule match dhoondhta hai
- **Bottom-up (Shift-Reduce):** Input scan karta hai jab tak rule match na ho, phir replace karta hai (stack use hota hai)

WebKit **Flex** (lexer generator) aur **Bison** (parser generator) use karta hai.

HTML Parsing — Special Case

HTML context-free grammar nahi hai (unlike CSS) kyunke:

1. HTML ki nature "**forgiving**" hai — tags omit ho sakte hain, browser implicitly add kar deta hai
2. Browsers traditionally invalid HTML ko tolerate karte hain
3. Parsing **reentrant** hoti hai — dynamic script (`document.write()`) parsing ke dauran naye tokens add kar sakti hai

Isi liye browsers **custom HTML parsers** banate hain, conventional top-down/bottom-up parsers use nahi karte.

HTML5 spec ke mutabiq parsing ke 2 stages:

1. **Tokenization** — lexical analysis, input ko tokens mein todna (start tags, end tags, attribute names/values) — ye ek **state machine** hai
2. **Tree Construction** — har token DOM element mein convert hota hai aur "stack of open elements" mein rakha jata hai (nesting mismatches/unclosed tags fix karne ke liye). Ye bhi state machine hai, states ko "**insertion modes**" kehte hain.

Error Tolerance Examples (WebKit):

- `</br>` ko `<br>` jaisa treat karta hai (IE/Firefox compatibility ke liye)
- **Stray table** (table ke andar table, lekin cell ke bahar) — WebKit inhe **sibling tables** bana deta hai
- **Nested forms** — dusra form ignore ho jata hai
- **Too deep tag hierarchy** — sirf 20 nested same-type tags allow hain, uske baad ignore
- Misplaced `</body>`/`</html>` — browser body tag kabhi close nahi karta khud se, `end()` call pe rely karta hai

Actions After Parsing Finish

Document "**interactive**" mark hota hai → deferred scripts execute hote hain → document "**complete**" ho jata hai → **load event** fire hota hai.

Unlike HTML, **CSS context-free grammar** hai — conventional parsers se parse ho sakti hai. WebKit Flex/Bison se generate karta hai, Firefox manually top-down parser use karta hai. Har CSS file ek **StyleSheet object** mein parse hota hai jo rules contain karta hai (selectors + declarations).

Scripts vs Style Sheets — Processing Order

- **Scripts:** Web synchronous model follow karta hai — jab parser `<script>` tag pe pohanche, parsing **ruk jati hai** jab tak script execute na ho jaye. External script ho tou fetch bhi synchronous hoti hai.
 - `defer` attribute — document parse hone ke baad execute hoga
 - HTML5 `async` — alag thread pe parallel parse/execute
 - **Speculative Parsing:** Script execute hote waqt ek doosra thread baaqi document parse karke pata lagata hai konse resources (external scripts, stylesheets, images) load karne hain, taake wo parallel connections pe load ho sakein (DOM tree modify nahi karta)
- **Style Sheets:** DOM tree change nahi karti isliye theoretically wait ki zarurat nahi. Lekin scripts style info maang sakti hain parsing ke dauran (agar unloaded ho tou wrong answer milega). Firefox: jab tak stylesheet load na ho, **saari scripts block** kar deta hai. WebKit: sirf tab block karta hai jab script un CSS properties ko access kare jo affected ho sakti hain.

Render Tree Construction

- Render tree ka DOM tree se **one-to-one relation nahi** hota:
 - Non-visual elements (`<head>`) render tree mein nahi aatay
 - `display: none` wale elements bhi nahi aatay (lekin `visibility: hidden` waale aatay hain)
 - Kuch DOM elements ke multiple renderers ho sakte hain (e.g. `<select>` ke 3 renderers: display area, dropdown, button)

- Floats/absolutely-positioned elements DOM tree ki jagah se different jagah render tree mein hotay hain (**placeholder frame** unki asal jagah pe hota hai)
- **WebKit RenderObject class:** har renderer ke paas `layout()`, `paint()` methods hotay hain, aur DOM node, computed style, containing layer ka reference.
- Element ka **box type** uske CSS `display` property se decide hota hai (block, inline, inline-block, list-item, none)

Style Computation — Difficulties & Optimizations

3 challenges:

1. Style data bohat bara structure hai → memory issue ho sakta hai
2. Har element ke liye matching rules dhoondhna performance-heavy hai (complex selectors)
3. Complex cascade rules apply karna

WebKit Style Sharing: Sibling/cousin nodes agar 10 conditions match karein (same tag, same class, same mouse/link/focus state, koi ID nahi, koi inline style nahi, koi sibling selector nahi, waghera) tou wo **same style object** share kar sakte hain.

Firefox Rule Tree + Context Tree: Do extra trees banati hai — matched rules ek tree mein store hotay hain (lazy computation), similar paths (jaise lexicon ke words) reuse hotay hain, structs (border, color jaisi categories) cache hotay hain taake dobara compute na karna pare. Structs 2 type ke hotay hain — **inherited** (parent se milte hain agar khud define na ho) aur **reset/non-inherited** (default value use hoti hai).

Manipulating Rules for Easy Match: Style sheet parse hone ke baad rules **hash maps** mein daale jate hain — by ID, by class, by tag name, aur ek general map. Isse matching mein 95%+ irrelevant rules eliminate ho jate hain.

Cascade Order (Low to High Priority)

1. Browser declarations

2. User normal declarations
3. Author normal declarations
4. Author important declarations
5. User important declarations

Specificity Formula

Specificity = **a-b-c-d** jahan:

- **a** = 1 agar inline style attribute hai, warna 0
- **b** = ID attributes ki count
- **c** = other attributes + pseudo-classes ki count
- **d** = element names + pseudo-elements ki count

Example: `ul ol li.red {}` → a=0, b=0, c=1, d=3 → specificity `(0,0,1,3)`

Sorting: WebKit small lists ke liye **bubble sort**, badi lists ke liye **merge sort** use karta hai.

Layout / Reflow

- **Dirty Bit System:** Jo bhi renderer change/add ho, khud ko aur children ko "dirty" mark karta hai (layout ki zarurat hai). Do flags: "dirty" aur "children are dirty".
- **Global Layout:** Poore render tree pe trigger hota hai (font size change jaisi global style change, ya screen resize)
- **Incremental Layout:** Sirf dirty renderers layout hotay hain — asynchronously trigger hota hai
- **Synchronous Layout:** Jab script `offsetHeight` jaisi property maange (turant layout chahiye hota hai)

Layout Process Pattern:

1. Parent apni width determine karta hai

2. Parent children pe iterate karta hai — position set karta hai (x, y), zaroorat ho tou child layout call karta hai (jo height calculate karta hai)
 3. Parent children ki heights + margins/padding se apni height set karta hai
 4. Dirty bit false set karta hai
- Rendering engine **single-threaded** hai (Firefox/Safari mein main thread, Chrome mein tab process ka main thread). Sirf network operations parallel threads pe hotay hain (usually 2-6 parallel connections).
 - **Event Loop:** Browser ka main thread infinite loop hai jo events (layout, paint) ka wait karta hai aur process karta hai.

Painting

- **Painting Order (CSS2 spec):** background color → background image → border → children → outline
- **Global vs Incremental Painting:** Global = poora tree paint, Incremental = sirf changed rectangle invalidate ho kar repaint hota hai ("dirty region")
- **Firefox Display List:** Ek baar tree traverse karke saare relevant renderers sahi paint order mein list mein rakhta hai (efficient repaint)
- **WebKit:** Purana rectangle bitmap mein save karta hai, sirf **delta** (farq) repaint karta hai

Positioning Schemes (CSS2 Visual Model)

Scheme	Behavior
Static	Normal flow, no explicit position
Relative	Normal position se required delta (offset) move hota hai, doosray elements affect nahi hotay

Scheme	Behavior
Float	Pehle normal flow mein layout, phir jitna ho sakay left/right shift — text uske around wrap hota hai
Absolute	Container (nearest positioned ancestor) ke relative position, normal flow se bahar
Fixed	Viewport ke relative, scroll karne pe bhi move nahi hota

Z-index / Layering: Boxes "**stacking contexts**" mein divide hotay hain — back elements pehle paint, forward elements upar. `z-index` property stacking order define karti hai.

6. Frontend Web Development Stack

3 Core Languages

Language	Role
HTML	Content add karta hai (Hypertext Markup Language)
CSS	Styling (Cascading Style Sheets) — colors, sizes, spacing
JavaScript	Interactivity — page ko dynamic banata hai

HTML Deep Points

- **Element** = opening tag + closing tag + content
- **Attributes** element ka behavior change karte hain (e.g. `type="text"` vs `type="number"`)
- **Anchor tag** (`<a>`) — "hyper" text ka asal matlab — pages ko link karta hai
- Elements nested ho kar **DOM (Document Object Model)** banate hain

- Page 2 parts mein split hoti hai: `<head>` (invisible metadata, title) aur `<body>` (visible content)
- Semantic tags browsers/bots ko meaning batatay hain — search engines aur **accessibility** (screen readers) ke liye zaroori
- `<div>` — generic sectioning element

CSS Deep Points

- **Inline style** (`style` attribute) — sirf ek element pe apply hota hai
- **Selectors** — kis element ko target karna hai decide karte hain (tag, class, id)
- Classes se ek style multiple elements pe reusable hoti hai
- **Specificity rules** decide karte hain konsi style win karegi jab conflict ho
- External stylesheet (linked in `<head>`) — most common approach

Box Model: Har element ek "box" hai jiske around **padding, border, margin** hotay hain.

- **Display: block** — poori horizontal space leta hai (e.g. headings)
- **Display: inline** — side by side line up hota hai (e.g. images)
- **Position property:** relative, absolute, fixed (jaisa upar detail mein cover ho chuka)

Responsive Design Tools:

- **Media Queries** — device info ke mutabiq alag styles apply karna
- **Flexbox** (`display: flex`) — rows/columns control karna (parent → children flow)
- **Grid** (`display: grid`) — multiple rows/columns ek sath control

CSS Turing-complete nahi hai lekin `calc()` (math operations) aur **custom properties** (variables) support karti hai. **SASS** jaisay preprocessors extra programmatic features add karte hain.

JavaScript Deep Points

- Script tag ke andar likha jata hai, ya external file se `src` attribute ke zariye link hota hai
- `defer` attribute — DOM load hone ke baad script run ho
- Dynamically typed language — koi type annotation zaroori nahi (isliye **TypeScript** use hota hai static typing ke liye)
- Variables: `let` (reassignable), `const` (non-reassignable)
- **Events:** click, mouse move, form input change — browser APIs (jaise `document.querySelector`) se elements grab karke **event listeners** assign kiye jate hain
- **Object** JavaScript ka fundamental data structure hai (dictionary/hashmap jaisa) — sab non-primitive types isi se inherit karte hain
- **Prototypal Inheritance:** Object clone ho kar ancestor chain banata hai (class-based inheritance se different — JS classes bhi actually syntactic sugar hain prototypal inheritance ke upar)

Frontend Frameworks

React, Vue, Svelte, Angular — sab UI ko **component tree** ki tarah represent karte hain. Component HTML+CSS+JS ko encapsulate karta hai, custom HTML element jaisa lagta hai. Ye **declarative** code produce karte hain (UI kya karna chahiye describe karta hai) jo **imperative** vanilla JS se easier hota hai.

Supporting Tools

Tool	Kaam
Bundler (Webpack)	Multiple JS/CSS files ko ek file mein combine karta hai browser ke liye

Tool	Kaam
Transpiler (TypeScript)	Enhanced JS likhne dete hain, phir normal JS mein convert karte hain
CSS Preprocessor (SASS)	Bundler + Transpiler jaisa hi lekin CSS ke liye — files organize karna aur enhanced CSS likhna
CSS Framework (Bootstrap)	Pre-written CSS code jo common problems solve karta hai
Linters (ESLint)	Code style guidelines follow na hone pe warn karta hai

Frontend-Backend Communication

- Purana tareeqa: **XMLHttpRequest**
- Modern tareeqa: **fetch** ya **axios** — backend ko URL pe message bhejtay hain (jaise `POST /create-order`)

7. Backend Web Development

Client-Server Model

- **Client** = message bhejne wala computer
- **Server** = message receive karne wala computer
- Computer ko messages receive karne ke liye **backend programming language** se program karna padta hai

Backend Languages & Frameworks

Language	Popular Framework
JavaScript (Node.js)	Express.js
Python	Django
Ruby	Ruby on Rails
Java	Spring

Package Managers

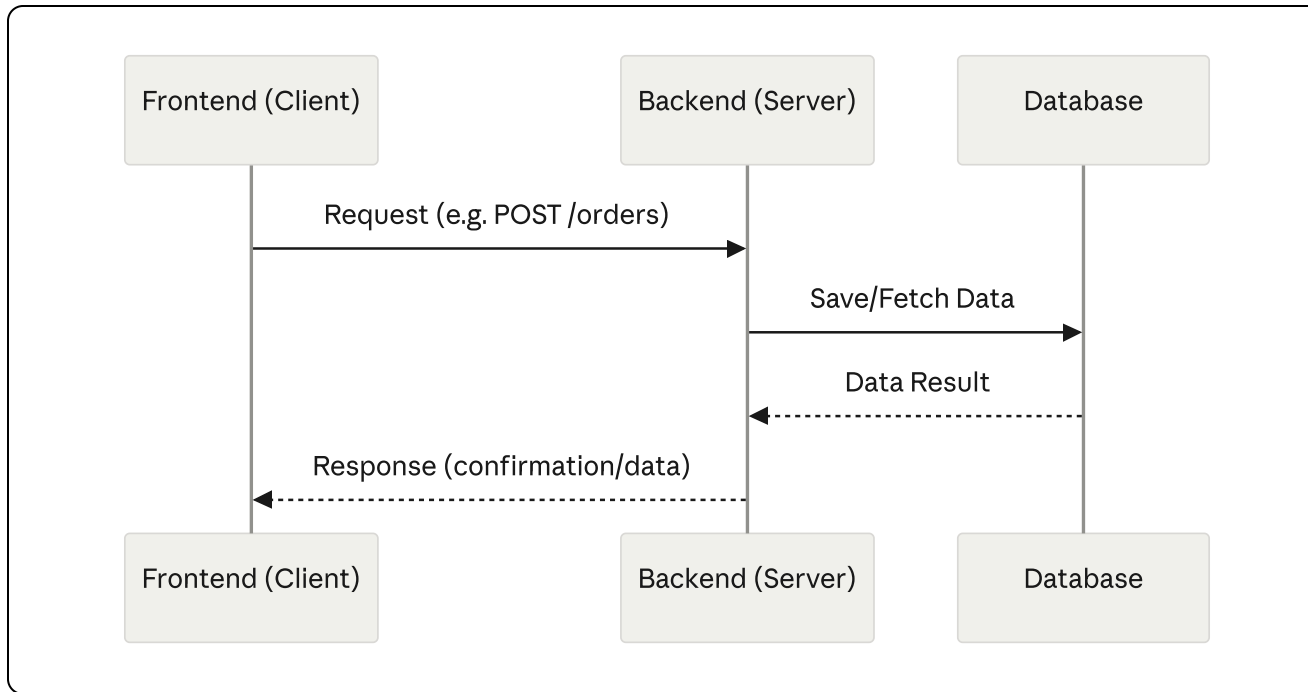
Language	Package Manager
JavaScript	NPM
Python	PIP
Ruby	Bundler
Java	Maven

Database

Data store/manage karne ke liye — alag computer pe run hoti hai, backend usse communicate karta hai.

- Popular: **MySQL, Postgres, MongoDB**

Request-Response Cycle



- **Request anatomy:** Type (GET/POST/PUT/DELETE) + Domain + URL Path
- **API (Application Programming Interface):** Backend jo request types allow karta hai unki list — agar API mein defined na ho tou error milta hai

REST — Naming Convention

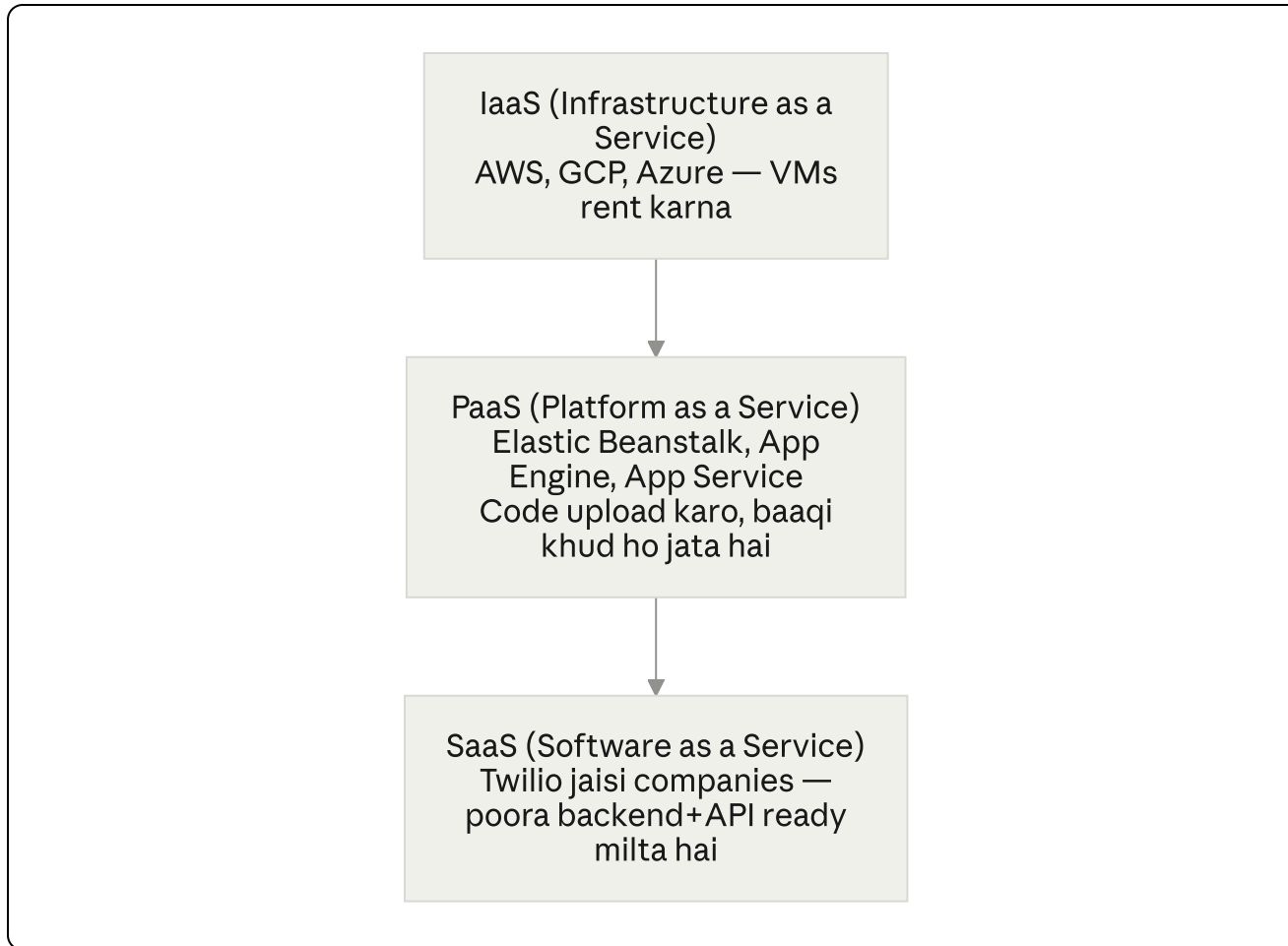
REST (Representational State Transfer) mein request ka **type** special meaning rakhta hai:

- **POST** = create karna (e.g. `POST /orders`)
- **GET** = retrieve karna
- **DELETE** = delete karna
- **PUT/PATCH** = update karna

Doosre conventions:

- **GraphQL** — sab requests (POST /graphql) pe jati hain
- **RPC** — (POST) + descriptive path (/create-order, /get-order-history)

Cloud Computing — 3 Foundations



- **IaaS:** Companies apne physical computers kharidne ki bajaye **VMs (Virtual Machines)** rent karti hain cloud providers se (AWS, GCP, Azure).
- **Load Balancer:** Traffic spike (jaise holiday season) handle karne ke liye multiple VMs pe same backend chalte hain, load balancer traffic evenly distribute karta hai.

- **PaaS:** Backend code upload karo, VMs/load balancer setup khud ho jata hai (Elastic Beanstalk for AWS, App Engine for GCP, App Service for Azure).
- **SaaS:** Kisi doosri company ka ready-made backend+API use karna (jaise email ke liye Twilio) — apna microservice banane ki zarurat nahi.

Microservices

Bara backend (millions of lines of code) ko chote independent backends mein split karna (e.g. orders service, payment service, email service) — har microservice apni language/database use kar sakti hai. Ek service dusri ko request bhej kar communicate karti hai.

Additional Backend Technologies

Need	Technology
Image storage	Blob Store (AWS S3) + CDN (CloudFront)
Text search	Search Database (Elastic Search)
Performance boost	Cache (Redis)
Data science	Analytical Database (Snowflake)
Scheduled tasks	Job Queue (RabbitMQ)

8. APIs — Deep Dive

API (Application Programming Interface) = ek contract/protocol jo define karta hai do applications aapas mein kaise communicate karein (requests aur responses ke zariye).

Restaurant Analogy

- **Restaurant** = ek application jo service provide karti hai (khana khilana)
- **Aap** = application jo service maang rahi hai
- **Customer service rep = API** — aap restaurant ke internal operations (kitchen capacity, staff, other reservations) mein diye bina bhi request kar sakte hain (reservation change)
- Bina API ke, आपको खुद restaurant के internal messy details handle करने पड़ता है — जो ना आपकी expertise है, ना restaurant ये private data reveal करना चाहेगा

Real Example — Apple Weather App

Apple खुद weather monitoring stations नहीं बनाता — वो weather.com जैसी service की **API** use करता है, जो उन्हें सिर्फ उतनी ही data देती है जितनी allow की गयी हो.

Web APIs — Working

- Request/Response usually **JSON** या **XML** format में होती है, internet के ज़रिये
 - Ek request-response cycle = **1 API Call**
 - Request में होता है: **Server endpoint URL + Request method** (HTTP method जो desired action बताता है)
 - Response में होता है: **Status code, Header, Response body**
 - Common status code example: **404** = URL not found
-

PART 2 — ACTIVE RECALL QUESTION BANK

(Answers ना देखें — पहले खुद attempt करें)

Section A — Internet Basics

A1. Internet को "network of networks" क्यों कहा जाता है? A2. Internet किस department ने

aur kyun banaya tha? A3. IP aur TCP mein kya farq hai? A4. Internet aur World Wide Web mein kya farq hai? A5. Socket kya hota hai aur ye Port se kaise different hai?

Section B — DNS

B1. DNS ke 4 server types kaunse hain aur unka role kya hai (library analogy ke sath explain karein)? B2. Recursive resolver aur authoritative server mein bunyadi farq kya hai? B3. DNS lookup ke 8 steps sequence mein likhein. B4. DNS caching kin 3 levels pe hoti hai? B5. Recursive, Iterative aur Non-recursive query mein farq batayein.

Section C — HTTP

C1. HTTP/0.9 mein kaunsa method sirf allowed tha? C2. HTTP/1.0 ki sabse badi limitation kya thi aur kyun? C3. Three-way handshake ke 3 steps kya hain? C4. HTTP/1.1 mein "Persistent Connections" aur "Pipelining" mein kya farq hai? C5. Chunked Transfer Encoding kab use hoti hai aur kaise kaam karti hai? C6. Head-of-Line Blocking kya hai aur HTTP/1.1 mein kyun hoti thi? C7. SPDY ka core performance idea kya tha (bandwidth vs latency)? C8. HTTP/2 ke 6 major features list karein. C9. HTTP/2 mein Stream ID odd/even kyun hote hain? C10. HPACK header compression kaise kaam karti hai? C11. Server Push kaise kaam karta hai (PUSH_PROMISE)? C12. HTTP/2 mein RST_STREAM ka fayda HTTP/1.1 ke comparison mein kya hai?

Section D — Web Hosting

D1. Shared hosting kya hoti hai aur ye kyun popular hai? D2. Jab aap hosting kharidte hain, actually kya kharidte hain? D3. Data center kin services provide karta hai? D4. cPanel ka kaam kya hai?

Section E — Browser Internals

E1. Browser ke 7 main components kya hain? E2. Rendering engine ka main flow (HTML se paint tak) steps mein likhein. E3. HTML context-free grammar kyun nahi hai (3 reasons)? E4. Lexer/Tokenizer aur Parser mein kya farq hai? E5. HTML parsing ke 2 stages kya hain? E6. WebKit "stray table" ko kaise handle karta hai? E7. CSS parsing HTML parsing se kaise different hai (context-free grammar ke lehaz se)? E8. Style

Sharing ke liye WebKit kin conditions ko check karta hai (koi 4 batayein)? E9. Firefox ka "Rule Tree" concept kya solve karta hai? E10. Cascade order (low to high) 5 levels mein likhein. E11. Specificity ka formula (a-b-c-d) explain karein. E12. Dirty Bit System kya hai? E13. Global layout vs incremental layout mein farq batayein. E14. Layout process ke 4 steps kya hain (parent-child pattern)? E15. Painting order (CSS2) kya hai? E16. Rendering engine single-threaded kyun hoti hai, aur network operations kis tarah alag hoti hain? E17. Float positioning aur Absolute positioning mein farq batayein. E18. Render tree ka DOM tree se one-to-one relation kyun nahi hota (2 examples dein)?

Section F — Frontend Stack

F1. HTML, CSS, JS mein se kaunsi language "interactivity" deti hai? F2. Box Model kya hota hai? F3. Display: block vs display: inline mein farq batayein. F4. Flexbox aur Grid mein basic farq kya hai? F5. Bundler aur Transpiler mein farq batayein (webpack vs typescript). F6. Prototypal inheritance kya hai? F7. Frontend frameworks (React/Vue) declarative code kyun produce karte hain? F8. SASS kis category ka tool hai (bundler+transpiler combined equivalent)?

Section G — Backend Stack

G1. Client-Server model mein client kya karta hai, server kya karta hai? G2. REST mein POST, GET, DELETE ka kya matlab hota hai? G3. GraphQL aur RPC, REST se kaise different hain (URL pattern ke lehaz se)? G4. IaaS, PaaS, SaaS mein farq examples ke sath batayein. G5. Load Balancer ka kaam kya hai? G6. Microservices kyun use ki jati hain? G7. Redis kis kaam ke liye use hota hai? G8. Job Queue (RabbitMQ) kis scenario mein use hoga?

Section H — APIs

H1. API ka full form aur simple definition kya hai? H2. Restaurant analogy mein "customer service rep" kis cheez ko represent karta hai? H3. Ek API call mein Request kin cheezon pe mushtamil hoti hai? H4. Response body mein kya hota hai? H5. 404 status code ka matlab kya hai?

PART 3 — ANSWER KEY

Section A

A1. Kyunke internet chote-chote individual networks (ghar, office, ISPs) ka aapas mein connect hona hai — network of networks. A2. US Department of Defense ne 1960s ke late mein banaya, taake ek decentralized network ho jo nuclear attack mein bhi survive kar sake. A3. IP packets ko sahi destination tak **route** karta hai (addressing), TCP unhe **reliable order mein deliver** karna ensure karta hai (packets break/reassemble, error checking). A4. Internet = hardware infrastructure (network of machines/routers). WWW = software layer jo internet ke upar chalti hai, HTTP se pages access hoti hain. A5. Socket = IP address + Port number ka combination (ek specific communication endpoint). Port sirf application/service identify karta hai device pe; Socket us port ko ek specific IP se jodta hai.

Section B

B1. Recursor (librarian — additional requests karta hai), Root (library index — aage direct karta hai), TLD (specific rack — domain ka last part e.g. .com), Authoritative (dictionary — final IP address deta hai). B2. Recursive resolver query ke **start** pe hota hai (record dhoondhta hai, multiple requests karta hai, cache karta hai). Authoritative server query ke **end** pe hota hai (actual record ka source of truth, khud jawab deta hai). B3. 1) User query → recursive resolver, 2) Resolver → root nameserver, 3) Root → TLD server address deta hai, 4) Resolver → TLD query, 5) TLD → domain nameserver address, 6) Resolver → authoritative nameserver query, 7) Authoritative → IP address return, 8) Resolver → browser ko IP deta hai. B4. Browser DNS cache → OS-level caching (stub resolver) → ISP recursive resolver cache. B5. Recursive = server ko poora answer ya error dena hota hai. Iterative = server best possible answer deta hai, ya referral deta hai next server ka. Non-recursive = server ke paas record already hai (cache/authoritative se) isliye further query ki zarurat nahi.

Section C

C1. Sirf **GET** method allowed tha. C2. Har request ke liye naya TCP connection khulta tha aur fulfil hone ke baad close ho jata — multiple resources (images, CSS, JS) ke liye bohat saari separate connections chahiye hoti thi jo three-way handshake overhead ki wajah se slow thi. C3. SYN (client random number x bhejta hai) → SYN-ACK (server apna y aur $x+1$ bhejta hai) → ACK (client $y+1$ bhejta hai). C4. Persistent Connections = connection default mein open rehti hai (close nahi hoti automatically), multiple sequential requests ja sakti hain. Pipelining = client bina response wait kiye multiple requests bhej sakta hai same connection pe, server same order mein response deta hai. C5. Jab server ko pehle se Content-Length pata nahi hota (dynamic content). Data chunks mein bheja jata hai, har chunk apna length batata hai, aakhir mein zero-length chunk transmission complete signal karta hai (`Transfer-Encoding: chunked` header ke sath). C6. Jab ek slow/heavy request pipeline mein phans jaye, uske peeche wali sab requests wait karti hain — kyunke HTTP/1.1 mein ek waqt sirf ek outstanding connection hoti thi aur responses same sequence mein aani zaroori thi. C7. Bandwidth badhane se performance ek had ke baad plateau ho jati hai, lekin latency kam karne se continuous performance gain milta hai — isliye SPDY ne latency reduce karne pe focus kiya. C8. Binary Protocol, Multiplexing, Header Compression (HPACK), Server Push, Request Prioritization, Security (TLS). C9. Client-initiated requests **odd** numbers use karte hain, server responses **even** numbers use karte hain — taake stream IDs unique aur predictable rahein. C10. Literal header values Huffman coding se encode hoti hain, aur client-server dono ek headers table maintain karte hain jisse repetitive headers (jaise user-agent) dobara poora bheje bina reference se identify ho jaate hain. C11. Server janta hai client agay konsa resource maangega, tou wo PUSH_PROMISE frame bhejta hai (stream se associated, promised stream ID ke sath) taake client ko batay "ye aa raha hai, mangna mat" — phir wo resource push kar deta hai. C12. HTTP/1.1 mein response rokne ke liye poori connection close karni parti thi (naya connection banana padta agli request ke liye). HTTP/2 mein RST_STREAM se sirf specific stream abort hoti hai, connection open rehti hai, baaqi streams chalti rehti hain.

Section D

D1. Ek hi server multiple websites host karta hai — sabse affordable/popular type hai kyunke resources share hotay hain. D2. Disk space (website files + emails) aur Bandwidth (monthly traffic allowance). D3. Network + power connectivity, temperature control, backup systems, fire suppression, physical security. D4. Email addresses setup, domains manage/forward, databases manage — technical complexity se shield karta hai user ko.

Section E

E1. User Interface, Browser Engine, Rendering Engine, Networking, UI Backend, JavaScript Interpreter, Data Storage. E2. HTML parse → DOM tree bane → CSS parse ho → Render Tree bane (DOM+style) → Layout (coordinates) → Paint (screen pe draw). E3. 1) Forgiving nature — tags omit ho sakte hain, 2) Browsers invalid HTML tolerate karte hain historically, 3) Parsing reentrant hai — script ke document.write() se dynamically tokens add ho sakte hain. E4. Lexer/Tokenizer input ko tokens (vocabulary) mein todta hai. Parser un tokens se grammar rules follow karke parse tree banata hai. E5. Tokenization (lexical analysis — tokens banana) aur Tree Construction (tokens se DOM elements banana aur stack mein rakhna). E6. Inner table ko outer table se **sibling** bana deta hai (stack se pop karke) — instead of nested table rehne dene ke. E7. CSS ek context-free grammar hai (conventional parsers se parse ho sakti hai) — HTML nahi hai (forgiving/error-tolerant nature ki wajah se). E8. Same mouse state, same tag name, same class attributes, koi ID nahi, koi inline style nahi, koi sibling selector nahi (in mein se koi bhi 4). E9. Ye repeated/similar computations avoid karne ke liye style values ko cache aur share karta hai (lexicon ke words ki tarah reuse hota hai) — memory aur computation dono save hoti hai. E10. Browser declarations → User normal → Author normal → Author important → User important (highest priority). E11. a (inline style = 1, warna 0) - b (ID count) - c (attributes + pseudo-classes count) - d (element names + pseudo-elements count) — concatenate karke specificity number banti hai. E12. Jab koi renderer change/add hota hai, khud ko aur children ko "dirty" mark karta hai taake pura layout dobara na ho, sirf zaroori parts layout hon. E13. Global layout poore render tree pe hoti hai (font-size change, resize jaisi cheezon se). Incremental layout sirf dirty renderers pe

hoti hai (asynchronously trigger). E14. 1) Parent apni width determine karta hai, 2) Parent har child ko position (x,y) deta hai aur zaroorat ho tou layout call karta hai, 3) Parent children ki heights se apni height set karta hai, 4) Dirty bit false set hoti hai. E15. Background color → Background image → Border → Children → Outline. E16. Almost sab kuch ek hi thread pe hota hai taake DOM manipulation consistent rahe — sirf network operations (jaise HTTP requests) parallel threads pe hoti hain (usually 2-6 connections) taake resources simultaneously load ho sakein. E17. Float: normal flow mein layout hota hai phir left/right shift hota hai, doosray elements uske around wrap hotay hain. Absolute: normal flow se completely bahar, container ke relative position specify hoti hai. E18. 1) `<head>` jaisay non-visual elements render tree mein nahi aatay, 2) `<select>` jaisay complex elements ke multiple renderers hotay hain (display area + dropdown + button).

Section F

F1. **JavaScript** — HTML/CSS sirf display karte hain, JS page ko interactive banata hai (DOM update karke). F2. Har element ek box hai jiske around padding, border, aur margin hotay hain. F3. Block apni poori horizontal space leta hai (top-to-bottom stack hota hai). Inline side-by-side line up hota hai (horizontal flow). F4. Flexbox ek dimension (row ya column) control karta hai. Grid dono dimensions (rows aur columns) ek sath control karta hai. F5. Bundler (webpack) multiple files ko combine/split karta hai. Transpiler (typescript) enhanced syntax ko normal JS mein convert karta hai jo browser samajh sakay. F6. Object clone ho kar ancestor chain banata hai jahan child apne ancestors ki properties/methods inherit karta hai — class-based inheritance se different mechanism. F7. Kyunke wo describe karte hain UI **kya** karni chahiye (end state), na ke **kaise** step-by-step DOM manipulate karna hai — jo imperative vanilla JS se easier hota hai. F8. Bundler + Transpiler combined equivalent — CSS files organize (bundle) karta hai aur enhanced CSS ko normal CSS mein transform (transpile) karta hai.

Section G

G1. Client message bhejta hai (request), server usay receive karta hai aur respond karta hai — server ko backend language se program karna padta hai messages receive karne

ke liye. G2. POST = create karna, GET = retrieve/fetch karna, DELETE = delete karna. G3. GraphQL sab requests ek hi endpoint ((POST /graphql)) pe bhejta hai. RPC POST + descriptive path use karta hai (/create-order). REST har action ke liye alag URL + HTTP method ka combination use karta hai. G4. IaaS = sirf VMs rent karna (AWS/GCP/Azure) — sab khud setup karna. PaaS = code upload karo, VM/load balancer setup khud ho jata hai (Elastic Beanstalk). SaaS = poora ready backend+API use karna (Twilio email service). G5. Multiple VMs pe traffic evenly distribute karta hai taake high traffic (holiday season) handle ho sakay. G6. Bara codebase chote independent services mein split hota hai — codebase manageable rehta hai, har service apni language/database choose kar sakti hai, ek service fail ho tou poora system down nahi hota. G7. Cache ke tor pe — primary database se stress kam karne ke liye, performance improve karta hai. G8. Jab koi task future mein schedule karna ho (e.g. subscription renewal se pehle email bhejna).

Section H

H1. Application Programming Interface — do applications ke darmiyan communicate karne ka tareeqa (requests aur responses ke zariye), bina internal implementation details expose kiye. H2. **API** ko represent karta hai — customer khud restaurant ke internal operations mein involve hue bina, rep (API) ke zariye request kar sakta hai. H3. Server endpoint URL + Request method (jo desired action batata hai, e.g. HTTP method). H4. Server resource jo client ko chahiye, ya application-specific messages — request ke hisab se vary karta hai. H5. **404 = URL Not Found** — requested resource server pe exist nahi karta.

Paper complete — total coverage: Internet fundamentals, DNS, HTTP (0.9 → HTTP/2), Web Hosting, Browser Internals (parsing, rendering, layout, paint), Frontend Stack, Backend Stack, aur APIs.